

1. Spring Framework Overview

1.1 Core Principles

- **Inversion of Control (IoC):**
 - A design principle where the control of object creation and management is transferred from application code to a container.
 - Promotes loose coupling between components.
- **Dependency Injection (DI):**
 - A specific form of IoC where dependencies are provided to a class rather than the class creating them itself.
 - Enhances testability and maintainability.

1.2 Modular Architecture

- The Spring Framework is modular, consisting of several modules that can be used independently or together.

Main Modules Include:

- **Core Container:** Provides IoC and DI features.
- **Beans:** Manages the configuration and lifecycle of beans.
- **Context:** Accesses application objects and supports internationalization, event propagation, and resource loading.
- **Data Access/Integration:** Supports data access and integration with various data sources.
- **Web:** Features for building web applications, including Web MVC, WebSocket, and Web Portlet.
- **AOP (Aspect-Oriented Programming):** Supports separation of cross-cutting concerns from business logic.
- **Test:** Supports testing of Spring components with JUnit or TestNG.

1.3 Key Features

- **Modularity:** Use only the parts needed.
 - **Aspect-Oriented Programming:** Improves code organization.
 - **Declarative Transaction Management:** Simplifies transaction management.
 - **Integration with Other Frameworks:** Seamless integration with Hibernate, JPA, JMS, etc.
 - **Flexible Configuration:** Supports XML, Java annotations, and Java-based configuration.
-

2. Spring MVC Architecture Overview

2.1 Key Components of Spring MVC

- **DispatcherServlet:**
 - Central component that handles all incoming requests and routes them to appropriate handlers (controllers).
- **Controller:**
 - Handles user requests and returns the appropriate response.

- Annotated with `@Controller` and methods are mapped using `@RequestMapping`.
- **Model:**
 - Represents the data of the application.
- **View Resolver:**
 - Resolves the view name returned by the controller to an actual view (e.g., JSP, Thymeleaf).
- **View:**
 - The presentation layer that renders the model data to the user.
- **Handler Mapping:**
 - Maps incoming requests to the appropriate controller methods.
- **Handler Adapter:**
 - Invokes the controller method and returns the model and view.
- **WebApplicationContext:**
 - A specialized application context for managing the lifecycle of beans in the web layer.

2.2 Spring MVC Request Processing Flow

1. **Request Handling:**
 - DispatcherServlet receives an HTTP request and consults HandlerMapping to find the appropriate controller.
2. **Controller Execution:**
 - The DispatcherServlet delegates the request to the selected controller, which processes the request and returns a logical view name.
3. **View Resolution:**
 - ViewResolver resolves the logical view name to an actual view.
4. **Response:**
 - The rendered view is returned to the client as an HTTP response.

Example of a Simple Spring MVC Application:

- **Controller:**

```

java
VerifyOpen In EditorRunCopy code
1@Controller
2@RequestMapping("/greeting")
3public class GreetingController {
4    @GetMapping
5    public String greeting(@RequestParam(name = "name", required = false,
6    defaultvalue = "world") String name, Model model) {
7        model.addAttribute("name", name);
8        return "greeting"; // View name
9    }
9}

```

- **View (greeting.jsp):**

```
jsp
VerifyOpen In EditorRunCopy code
1<h1>Hello, ${name}!</h1>
```

- **Configuration (web.xml):**

```
xml
VerifyOpen In EditorRunCopy code
1<servlet>
2    <servlet-name>dispatcher</servlet-name>
3    <servlet-class>org.springframework.web.servlet.DispatcherServlet</
servlet-class>
4    <load-on-startup>1</load-on-startup>
5</servlet>
6<servlet-mapping>
7    <servlet-name>dispatcher</servlet-name>
8    <url-pattern>/</url-pattern>
9</servlet-mapping>
```

3. Overview of Spring Framework Modules

3.1 Core Container Modules

- **Spring Core:**
 - Provides foundational features, including IoC and DI capabilities.
- **Spring Beans:**
 - Manages the configuration and lifecycle of beans.
- **Spring Context:**
 - Accesses application objects and supports features like event handling and internationalization.
- **Spring Expression Language (SpEL):**
 - Allows querying and manipulation of objects at runtime.

3.2 Data Access/Integration Modules

- **Spring JDBC:**
 - Simplifies database access and error handling using JDBC.
- **Spring ORM:**
 - Integrates with popular ORM frameworks like Hibernate and JPA.
- **Spring JMS:**
 - Provides support for Java Messaging Service (JMS).
- **Spring Transactions:**
 - Manages programmatic and declarative transaction management.

3.3 Web Modules

- **Spring Web:**

- Provides basic web-oriented integration features.
- **Spring Web MVC:**
 - Implements the MVC design pattern for building web applications.
- **Spring Web WebSocket:**
 - Supports WebSocket-based communication.
- **Spring Web Portlet:**
 - Provides support for building portlet applications.

3.4 AOP (Aspect-Oriented Programming) Module

- **Spring AOP:**
 - Provides support for aspect-oriented programming, allowing developers to define cross-cutting concerns separately from business logic.

3.5 Testing Module

- **Spring Test:**
 - Provides support for testing Spring components with JUnit or TestNG.

3.6 Miscellaneous Modules

- **Spring Security:**
 - Provides security features for authentication and authorization.
 - **Spring Batch:**
 - Supports batch processing and managing batch jobs.
 - **Spring Integration:**
 - Facilitates building message-driven and event-driven architectures.
 - **Spring Cloud:**
 - Provides tools for building cloud-native applications.
-

4. Spring 4 Annotations Overview

4.1 Core Annotations

- **@Component:**
 - Indicates that a class is a Spring component.
- **@Service:**
 - Specialization of @Component for service layer classes.
- **@Repository:**
 - Specialization of @Component for Data Access Objects (DAOs).
- **@Controller:**
 - Defines a controller in Spring MVC.

4.2 Dependency Injection Annotations

- **@Autowired:**
 - Used for automatic dependency injection.
- **@Qualifier:**
 - Specifies which bean to inject when multiple candidates are available.
- **@Value:**
 - Injects values from property files or environment variables.

4.3 Configuration Annotations

- **@Configuration:**
 - Indicates that a class declares one or more **@Bean** methods.
- **@Bean:**
 - Indicates that a method produces a bean to be managed by the Spring container.
- **@PropertySource:**
 - Specifies the location of properties files to be loaded.

4.4 Web Annotations

- **@RequestMapping:**
 - Maps web requests to specific handler methods.
- **@GetMapping:**
 - Shortcut for `@RequestMapping(method = RequestMethod.GET)`.
- **@PostMapping:**
 - Composed annotation for handling POST requests.
- **@ResponseBody:**
 - Indicates that the return value of a method should be bound to the web response body.

4.5 Additional Annotations

- **@Transactional:**
 - Indicates that a method or class should be executed within a transaction context.
- **@Aspect:**
 - Defines an aspect in AOP.
- **@EnableAspectJAutoProxy:**
 - Enables support for handling components marked with **@Aspect**.

5. Inversion of Control (IoC) in Spring Framework

5.1 Key Concepts of IoC in Spring

- **Dependency Injection (DI):**
 - A specific form of IoC where the framework provides dependencies to a class.

- **IoC Container:**
 - Manages the instantiation, configuration, and lifecycle of application objects (beans).

5.2 Types of IoC Containers in Spring

1. BeanFactory:

- The simplest container providing basic support for DI.
- Lazily initializes beans.

2. ApplicationContext:

- A more advanced container that eagerly initializes singleton beans and provides additional features.

5.3 How IoC Works in Spring

1. Configuration Metadata:

- Required to know how to create and manage beans.

2. Bean Definition:

- Each bean is defined in the configuration metadata.

3. Dependency Resolution:

- The IoC container resolves dependencies by injecting required objects.

4. Lifecycle Management:

- The container manages the lifecycle of beans, including instantiation and destruction.

Example of IoC in Spring:

- **Bean Class:**

```
java
VerifyOpen In EditorRunCopy code
1public class UserService {
2    private UserRepository userRepository;
3
4    public UserService(UserRepository userRepository) {
5        this.userRepository = userRepository;
6    }
7
8    public void addUser (String name) {
9        userRepository.save(name);
10    }
11}
```

- **Spring Configuration (XML):**

```
xml
VerifyOpen In EditorRunCopy code
1<beans>
2    <bean id="userRepository" class="com.example.UserRepository"/>
3    <bean id="userService" class="com.example.UserService">
4        <constructor-arg ref="userRepository"/>
5    </bean>
6</beans>
```

6. Dependency Injection in Spring Framework

6.1 Key Concepts of Dependency Injection

- **Inversion of Control (IoC):**
 - Control of object creation is transferred to the Spring container.
- **Beans:**
 - Objects managed by the Spring IoC container.
- **Configuration:**
 - Beans and their dependencies are defined using XML, Java annotations, or Java configuration classes.

6.2 Types of Dependency Injection in Spring

1. Constructor Injection:

- Dependencies are provided through a class constructor.
- Ensures required dependencies are available at object creation.

2. Setter Injection:

- Dependencies are set through setter methods after object creation.
- Useful for optional dependencies.

3. Field Injection:

- Dependencies are injected directly into fields.
- Less preferred due to lack of visibility.

Example of Constructor Injection:

```
java
VerifyOpen In EditorRunCopy code
1@Component
2public class ProductService {
3    private final ProductRepository productRepository;
4
5    @Autowired
6    public ProductService(ProductRepository productRepository) {
7        this.productRepository = productRepository;
8    }
9}
```

Example of Setter Injection:

```
java
VerifyOpen In EditorRunCopy code
1@Component
2public class OrderService {
3    private ProductService productService;
4
5    @Autowired
6    public void setProductService(ProductService productService) {
7        this.productService = productService;
8    }
9}
```

Example of Field Injection:

```
java
VerifyOpen In EditorRunCopy code
1@Component
2public class UserService {
3    @Autowired
4    private UserRepository userRepository; // Field Injection
5}
```

7. Spring Beans Explanation with Examples

7.1 Key Concepts of Spring Beans

- **Bean Definition:**
 - Defined in the Spring container using configuration metadata.
- **Scope:**
 - Defines the lifecycle of a bean (singleton, prototype, etc.).
- **Lifecycle:**
 - Managed by the Spring container, including instantiation, initialization, and destruction.

7.2 Defining Spring Beans

1. Using Annotations:

- **@Component:** Generic stereotype for any Spring-managed component.
- **@Service:** For service layer classes.
- **@Repository:** For Data Access Objects (DAOs).
- **@Controller:** For Spring MVC controllers.

Example:

```
java
VerifyOpen In EditorRunCopy code
1@Component
2public class MyComponent {
3    public void doSomething() {
4        System.out.println("Doing something...");
5    }
6}
```

2. Using XML Configuration:

```
xml
VerifyOpen In EditorRunCopy code
1<beans>
2    <bean id="myComponent" class="com.example.MyComponent"/>
3</beans>
```

7.3 Bean Scopes

- **Singleton:**
 - One instance per Spring container (default).
- **Prototype:**

- New instance created each time it is requested.
- **Request:**
 - New instance for each HTTP request.
- **Session:**
 - New instance for each HTTP session.

Example of Singleton and Prototype Scopes:

```
java
VerifyOpen In EditorRunCopy code
1@Component
2@Scope("singleton")
3public class SingletonBean { }
4
5@Component
6@Scope("prototype")
7public class PrototypeBean { }
```

8. Autowiring Beans in Spring Framework

8.1 Types of Autowiring

- **No Autowiring:**
 - Default mode where no autowiring is performed.
- **By Type:**
 - Spring looks for a bean of the same type as the property to be autowired.
- **By Name:**
 - Spring looks for a bean with the same name as the property.
- **Constructor:**
 - Spring uses the constructor of the bean to inject dependencies.
- **Autodetect:**
 - Tries to autowire by constructor first, then by type.

8.2 Enabling Autowiring

- **XML Configuration:**

```
xml
VerifyOpen In EditorRunCopy code
1<bean id="employee" class="com.example.Employee"/>
2<bean id="employeeService" class="com.example.EmployeeService"
autowire="byType"/>
```

- **Java Configuration:**

```
java
VerifyOpen In EditorRunCopy code
1@Configuration
2@ComponentScan(basePackages = "com.example")
3public class AppConfig {
4    @Bean
5    public Employee employee() {
```

```

6         return new Employee();
7     }
8
9     @Bean
10    public EmployeeService employeeService() {
11        return new EmployeeService();
12    }
13}

```

9. Bean Scopes in Spring Framework

9.1 Singleton Scope

- **Definition:**
 - A singleton-scoped bean is created only once per Spring IoC container.

Example:

```

java
VerifyOpen In EditorRunCopy code
1@Component
2public class SingletonBean {
3    public SingletonBean() {
4        System.out.println("SingletonBean instance created");
5    }
6}

```

9.2 Prototype Scope

- **Definition:**
 - A prototype-scoped bean is created each time it is requested.

Example:

```

java
VerifyOpen In EditorRunCopy code
1@Component
2@Scope("prototype")
3public class PrototypeBean {
4    public PrototypeBean() {
5        System.out.println("PrototypeBean instance created");
6    }
7}

```

9.3 Request Scope

- **Definition:**
 - A request-scoped bean is created for each HTTP request.

Example:

```

java
VerifyOpen In EditorRunCopy code
1@Component
2@Scope("request")
3public class RequestScopedBean {
4    public RequestScopedBean() {
5        System.out.println("RequestScopedBean instance created");
6    }
7}

```

9.4 Session Scope

- **Definition:**

- A session-scoped bean is created for each HTTP session.

Example:

```
java
VerifyOpen In EditorRunCopy code
1@Component
2@Scope("session")
3public class SessionScopedBean {
4    public SessionScopedBean() {
5        System.out.println("SessionScopedBean instance created");
6    }
7}
```

10. Spring MVC Overview

10.1 Key Components of Spring MVC

- **Model:**

- Represents the data and business logic of the application.

- **View:**

- Represents the user interface of the application.

- **Controller:**

- Acts as an intermediary between the model and the view.

10.2 Setting Up Spring MVC

- **Configuration:**

```
java
VerifyOpen In EditorRunCopy code
1@Configuration
2@EnableWebMvc
3@ComponentScan(basePackages = "com.example")
4public class WebConfig {
5    // Additional configuration can be added here
6}
```

- **Creating a Controller:**

```
java
VerifyOpen In EditorRunCopy code
1@Controller
2@RequestMapping("/greeting")
3public class GreetingController {
4    @GetMapping
5    public String greeting(Model model) {
6        model.addAttribute("message", "Hello, World!");
7        return "greeting"; // View name
8    }
9}
```

- **Creating a View:**

```
jsp
VerifyOpen In EditorRunCopy code
1<h1>${message}</h1>
```

- **Configuring View Resolvers:**

```
java
VerifyOpen In EditorRunCopy code
1@Bean
2public ViewResolver viewResolver() {
3    InternalResourceViewResolver resolver = new InternalResourceViewResolver();
4    resolver.setPrefix("/WEB-INF/views/");
5    resolver.setSuffix(".jsp");
6    return resolver;
7}
```

11. Spring Validation Example with Custom Validator

11.1 Setting Up the Spring Project

- Ensure necessary dependencies for Spring Web and Spring Validation are included in pom.xml.

11.2 Create a Custom Annotation

```
java
VerifyOpen In EditorRunCopy code
1@Constraint(validatedBy = AgeValidator.class)
2@Target({ ElementType.METHOD, ElementType.FIELD })
3@Retention(RetentionPolicy.RUNTIME)
4public @interface ValidAge {
5    String message() default "Age must be between 18 and 65";
6    Class<?>[] groups() default {};
7    Class<? extends Payload>[] payload() default {};
8}
```

11.3 Implement the Custom Validator

```
java
VerifyOpen In EditorRunCopy code
1public class AgeValidator implements ConstraintValidator<ValidAge, Integer> {
2    @Override
3    public boolean isValid(Integer age, ConstraintValidatorContext context) {
4        return age != null && age >= 18 && age <= 65;
5    }
6}
```

11.4 Create a Model Class

```
java
VerifyOpen In EditorRunCopy code
1public class Person {
2    @NotNull(message = "Name cannot be null")
3    private String name;
4
5    @ValidAge
6    private Integer age;
7
8    // Getters and Setters
9}
```

11.5 Create a Controller

```
java
VerifyOpen In EditorRunCopy code
1@RestController
2@RequestMapping("/api/person")
3public class PersonController {
4    @PostMapping
5    @ResponseStatus(HttpStatus.CREATED)
6    public Person createPerson(@Valid @RequestBody Person person) {
7        return person; // Logic to save the person
8    }
9}
```

11.6 Exception Handling

```
java
VerifyOpen In EditorRunCopy code
1@ControllerAdvice
2public class GlobalExceptionHandler {
3    @ExceptionHandler(MethodArgumentNotValidException.class)
4    public ResponseEntity<Map<String, String>>
5    handleValidationExceptions(MethodArgumentNotValidException ex) {
6        Map<String, String> errors = new HashMap<>();
7        ex.getBindingResult().getFieldErrors().forEach(error ->
8            errors.put(error.getField(), error.getDefaultMessage()));
9        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
10    }
11}
```

12. Spring i18n Internationalization Example

12.1 Project Structure

- Organize your project with a specific structure for controllers, configuration, resources, and views.

12.2 Create Message Properties Files

- **messages.properties** (default):

```
properties
VerifyOpen In EditorRunCopy code
1greeting.message=Hello, World!
```

- **messages_fr.properties** (French):

```
properties
VerifyOpen In EditorRunCopy code
1greeting.message=Bonjour, le monde!
```

12.3 Spring Configuration

```
java
VerifyOpen In EditorRunCopy code
1@Configuration
2public class WebConfig implements WebMvcConfigurer {
3    @Bean
4    public LocaleResolver localeResolver() {
5        return new CookieLocaleResolver();
6    }
7}
```

```

6    }
7
8    @Bean
9    public LocaleChangeInterceptor localeChangeInterceptor() {
10        LocaleChangeInterceptor interceptor = new LocaleChangeInterceptor();
11        interceptor.setParamName("lang");
12        return interceptor;
13    }
14
15    @Bean
16    public ResourceBundleMessageSource messageSource() {
17        ResourceBundleMessageSource messageSource = new
ResourceBundleMessageSource();
18        messageSource.setBasename("messages");
19        return messageSource;
20    }
21
22    @Override
23    public void addInterceptors(InterceptorRegistry registry) {
24        registry.addInterceptor(localeChangeInterceptor());
25    }
26}

```

12.4 Create a Controller

```

java
VerifyOpen In EditorRunCopy code
1@Controller
2public class GreetingController {
3    @Autowired
4    private MessageSource messageSource;
5
6    @GetMapping("/greeting")
7    public String greeting(HttpServletRequest request, Model model) {
8        Locale locale = request.getLocale();
9        String greetingMessage = messageSource.getMessage("greeting.message",
null, locale);
10        model.addAttribute("message", greetingMessage);
11        return "greeting"; // View name
12    }
13}

```

12.5 Create the View

```

jsp
VerifyOpen In EditorRunCopy code
1<h1>${message}</h1>
2<a href="?lang=en">English</a>
3<a href="?lang=fr">Français</a>

```

Conclusion The Spring Framework provides a comprehensive programming and configuration model for modern Java-based enterprise applications. Its architecture is designed around the principles of IoC and DI, promoting loose coupling and enhancing testability. With its modular structure, Spring allows developers to choose the components they need, making it a versatile choice for building robust applications.

If you have any specific questions or need further details on any aspect of the Spring Framework, feel free to ask!

